University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

TD6 Computer Security

StackX64 Project – Progressive x86-64 exploitation

Ethical and Legal Disclaimer

The techniques presented in this lab are studied within a **strictly academic context** in order to understand the mechanisms of memory vulnerability exploitation and thus be able to better protect against them.
Any use of these techniques outside the educational context is illegal and punishable under criminal law. Under French law (Chapter III of the Penal Code, articles 323-1 to 323-8 – *Offences against automated data processing systems*):

- Unauthorized access to or remaining in an automated data processing system (art. 323-1): **3 years** imprisonment and a fine of **€100,000**
- Obstructing the operation of an automated data processing system (art. 323-2): **5 years** imprisonment and a fine of **€150,000**
- Fraudulent introduction of data into an automated data processing system (art. 323-3): **5 years** imprisonment and a fine of **€150,000**

Penalties are increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (art. 323-4-1). *Penalties updated by the LOPMI Act no. 2023-22 of January 24, 2023.*

In this lab project, you will **progressively** explore exploitation techniques for memory vulnerabilities in x86-64. Starting from stack reconnaissance up to advanced attacks (shellcode, ret2libc, format strings for writing), each phase builds upon the previous one and introduces a new attack or defense technique.

Prerequisites: TD5 (simple buffer overflow, return address modification, format strings for reading). Basic knowledge of GDB and x86-64 assembly.

Required environment: x86-64 Linux machine (Debian 12/13 recommended). Tools: `gcc`, `gdb`, `python3`, `objdump`, `ROPgadget` (optional).

Warning

All experiments must be performed on a **dedicated virtual machine** or an isolated environment. Disable ASLR **only** in this environment:

```
sudo sysctl -w kernel.randomize_va_space=0
```

Exercise 1 (StackX64 Project)**.**

Phase 1 – Reconnaissance and memory mapping *(∼25 min)*

Course Reminder

The x86-64 stack and key registers:
In x86-64 architecture, the **stack** grows toward lower addresses. The main registers are:

- **RSP** (*Stack Pointer*): points to the top of the stack (last pushed value)
- **RBP** (*Base Pointer*): points to the base of the current function's stack frame
- **RIP** (*Instruction Pointer*): address of the next instruction to execute

Stack frame layout during a function call:

```
High addresses
+---------------------------+
| ...                       |
| Argument 7+ (if > 6 args)|  <-- pushed by the caller
+---------------------------+
| Return address (8 bytes)  |  <-- pushed by CALL
+---------------------------+
| Saved RBP (8 bytes)       |  <-- RBP points here
+---------------------------+
| Local variables           |  <-- RSP points to the bottom
| (buffer, etc.)            |
+---------------------------+
Low addresses
```

Essential GDB commands:

- `disassemble <func>`: disassemble a function
- `info registers`: display registers
- `x/Nx <addr>`: examine N memory words in hexadecimal
- `x/s <addr>`: examine a string
- `break *<addr>`: set a breakpoint at an address
- `run < <(python3 -c "...")`: run with input generated by Python

1a) Compile the program `stackx64.c` with **all protections disabled**:

```
gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
```

Run the program with a normal input (e.g., `AAAA`). Observe the addresses displayed by the `[DEBUG]` messages.

1b) Open the program in GDB (`gdb ./stackx64`). Disassemble the function `vulnerable_function` with `disassemble vulnerable_function`. Identify the instruction `sub rsp, 0x...` that reserves space for local variables. What is the reserved size?

1c) Set a breakpoint after the `gets` in `vulnerable_function`. Enter `AAAAAAAA` (8 A's). Examine the stack with `x/20gx $rsp`. Draw the complete stack diagram identifying:

- The address and contents of the buffer
- The address and value of the saved RBP
- The address and value of the return address

1d) Calculate the **offset** (distance in bytes) between the beginning of the buffer and the return address. Verify your calculation with GDB by sending a string of known size and observing the overwrite.

1e) Construct an input that redirects execution to `secret_function()` by overwriting the return address. Use Python3:

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*?? + b'\x...\x...\x
    ...\x...\x...\x...\x00\x00')"
```

Replace `??` with the offset and the `\x..` with the address of `secret_function` in **little-endian**.

Phase 2 – Shellcode analysis and testing *(∼30 min)*

Course Reminder

Shellcode: raw machine code
A **shellcode** is a sequence of bytes representing directly executable machine code, designed to be injected into memory. Its classic goal is to open a *shell* (`/bin/sh`).

x86-64 system call conventions (Linux):

Register	Role
`rax`	Syscall number (59 = `execve`)
`rdi`	1st argument (`filename`)
`rsi`	2nd argument (`argv`)
`rdx`	3rd argument (`envp`)

Null byte constraint: C string handling functions (`strcpy`, `gets`, etc.) stop at the first `0x00` byte. Therefore, the shellcode must contain **no null bytes**.

Here is a 25-byte x86-64 shellcode that executes `execve("/bin//sh", NULL, NULL)`:

```
xor     rsi, rsi                ; rsi = 0 (argv = NULL)
push    rsi                     ; push 0 (string terminator)
movabs  rdi, 0x68732f2f6e69622f ; rdi = "/bin//sh"
push    rdi                     ; push "/bin//sh" onto the stack
push    rsp                     ; push the stack top address
pop     rdi                     ; rdi = pointer to "/bin//sh"
xor     rdx, rdx                ; rdx = 0 (envp = NULL)
mov     al, 59                  ; rax = 59 (execve syscall number)
syscall                         ; execve system call
```

Shellcode execve – 25 bytes

Byte representation:

```
\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68
\x57\x54\x5f\x48\x31\xd2\xb0\x3b\x0f\x05
```

2a) For each instruction in the shellcode above, explain its role in constructing the `execve("/bin//sh", NULL, NULL)` call.

2b) Why do we use `xor rsi, rsi` instead of `mov rsi, 0` to zero a register? What problem would `mov rsi, 0` cause? (Hint: examine the opcodes with `objdump` or an online assembler.)

2c) Explain the concept of the **NOP sled** (`0x90` = `NOP` instruction). Draw a diagram showing why we aim at the middle of the NOP sled rather than exactly at the beginning of the shellcode.

2d) Compile and test the program `test_shellcode.c`:

```
gcc -z execstack -o test_shellcode test_shellcode.c
./test_shellcode
```

What happens? Type `exit` to quit the obtained shell.

Phase 3 – Shellcode injection via overflow *(∼35 min)*

Course Reminder

Structure of a shellcode injection payload:
The payload is constructed to fit exactly within the buffer + RBP + return address space:

```
+---------------------------------------------+----------+
| NOP sled (0x90 * N) | Shellcode (25 bytes)   | Ret addr |
+---------------------------------------------+----------+
|<------------ 72 bytes (offset) ------------>|<--8 b.-->|
                                                Total: 80 bytes
```

The return address points to an address **inside the NOP sled** (within the buffer).

Warning

ASLR must be **disabled** for this phase. Verify with:

```
cat /proc/sys/kernel/randomize_va_space
```

The value must be `0`. Otherwise: `sudo sysctl -w kernel.randomize_va_space=0`

3a) In GDB, find the exact address of the buffer. Set a breakpoint just after the `gets`, run with any input, then examine the address displayed by the program or read the RSP register.

3b) Build the complete Python3 payload:

- **NOP sled**: 47 bytes of `\x90`
- **Shellcode**: 25 bytes (given in Phase 2)
- **Return address**: 8 bytes pointing to the NOP sled (in little-endian)
- Total: $47 + 25 + 8 = 80$ bytes

Write the Python3 script that generates this payload.

3c) Run the exploit:

```
(python3 exploit_phase3.py ; cat) | ./stackx64
```

The `; cat` keeps `stdin` open to interact with the obtained shell. Verify that you get a shell by typing `whoami`.

3d) Explain why the return address must be written in **little-endian**. Convert the address `0x7fffffffde10` to little-endian.

Warning

Addresses may be **different** between GDB and direct execution. GDB adds environment variables that shift the stack. If the exploit works in GDB but not outside, adjust the address (typically a few dozen bytes of difference).

Phase 4 – NX protection and Return-to-libc *(∼35 min)*

Course Reminder

NX (No-eXecute) / DEP (Data Execution Prevention):
The **NX** protection marks the stack (and other data regions) as **non-executable**. Any attempt to execute code on the stack causes a `SIGSEGV` signal.

Return-to-libc (ret2libc):
Instead of injecting code, we **reuse** functions already present in memory (in the libc). The call `system("/bin/sh")` requires:

1. A **gadget** `pop rdi; ret` to place the argument in RDI
2. The address of the string `"/bin/sh"` in the libc
3. The address of the `system()` function

16-byte stack alignment:
The x86-64 ABI requires that RSP be aligned to 16 bytes during a `call`. After a `ret` (which pops 8 bytes), RSP is no longer aligned. An additional `ret` gadget realigns the stack.

ret2libc payload structure:

```
+--------+-----+------------+----------+----------+
| Padding| ret | pop rdi;ret| "/bin/sh" | system() |
| 72 b.  |8 b. | 8 bytes    | 8 bytes  | 8 bytes  |
+--------+-----+------------+----------+----------+
           ^       ^              ^          ^
           |       |              |          |
         align   gadget      argument    function
```

4a) Recompile `stackx64.c` **without** `-z execstack` (NX enabled):

```
gcc -fno-stack-protector -no-pie -g stackx64.c -o stackx64
```

Re-run the Phase 3 exploit. What happens and why?

4b) Find the address of `system()` in GDB:

```
(gdb) break main
(gdb) run
(gdb) print system
```

4c) Find the address of the string `"/bin/sh"` in the libc:

```
strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

Then add the found offset to the libc base address (visible with `info proc mappings` in GDB).

4d) Find a `pop rdi; ret` gadget in the binary or the libc:

```
ROPgadget --binary stackx64 | grep "pop rdi"
```

Or with `objdump`:

```
objdump -d stackx64 | grep -A1 "pop.*rdi"
```

4e) Find a simple `ret` gadget (for alignment) in the binary.

4f) Build the complete ret2libc payload in Python3 and run the exploit.

4g) Fill in the comparative table between shellcode injection (Phase 3) and ret2libc (Phase 4):

Criterion	**Shellcode injection**	**Return-to-libc**
Code injected?		
Works with NX?		
Requires `-z execstack`?		
Payload flexibility		
Construction complexity		
Sensitive to libc ASLR?		

Phase 5 – Format string write attack *(∼30 min)*

Course Reminder

The %n specifier: from reading to writing
In TD5, you used `%x` and `%p` to **read** the stack via `printf(buffer)`. The `%n` specifier goes further: it **writes** the number of characters printed so far to the address pointed to by the corresponding argument.

Variants of %n:

Format	Write size	Usage
`%n`	4 bytes (int)	Full write
`%hn`	2 bytes (short)	16-bit write
`%hhn`	1 byte (char)	8-bit write

Direct access with $: The format `%k$hhn` uses the **k-th** stack argument for writing, without consuming previous arguments.

64-bit difficulty: x86-64 addresses like `0x00000000004040XX` contain **null bytes**. If the address is at the beginning of the string, `printf` stops at the first `0x00`. Solution: place the address **at the end of the string** and adjust the offset `k`.

Warning

Compile `formatx64.c` with:

```
gcc -fno-stack-protector -no-pie -U_FORTIFY_SOURCE -g formatx64.c -o
    formatx64
```

The `-U_FORTIFY_SOURCE` option disables the protection that blocks `%n` in recent versions of the glibc.

5a) Locate the buffer on the stack using `%p`:

```
./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p"
```

Find the offset `k` such that `%k$p` displays `0x4141414141414141` (the 8 `A`'s = `0x41`).

5b) Find the address of the global variable `is_admin` with `nm` or `objdump`:

```
nm formatx64 | grep is_admin
```

5c) Build the `%hhn` exploit to write a non-zero value to `is_admin`. The strategy is:

(a) Place the address of `is_admin` at the end of the payload (after the format specifiers)

(b) Use padding to adjust the number of printed characters

(c) Use `%hhn` with the appropriate offset to write to `is_admin`

5d) Run the exploit and verify that the message `"Admin access granted!"` is displayed.

Phase 6 – Synthesis and defense mechanisms *(∼25 min)*

Course Reminder

Defense in depth:
No single protection blocks all attacks. Security relies on **multiple complementary layers**:

Mechanism	Principle	GCC Flag
Stack canaries	Sentinel value before the return address	`-fstack-protector`
NX / DEP	Non-executable stack	(enabled by default)
ASLR	Memory address randomization	(kernel)
PIE	Executable code at random address	`-pie`
Full RELRO	Read-only GOT	`-Wl,-z,relro,-z,now`

6a) For each of the 5 defense mechanisms (stack canaries, NX, ASLR, PIE, Full RELRO), describe in 2–3 sentences:

- Its **operating principle**
- Which **attack(s)** it blocks
- The **flag** (compilation or system) to enable/disable it

6b) Fill in the following matrix (5 protections × 4 attacks). For each cell, indicate: **BLOCKS**, **does not block**, or **partially** (with justification).

	Overflow → secret()	**Shellcode injection**	**Ret2libc**	**Format string %n**
Stack canaries				
NX / DEP				
ASLR				
PIE				
Full RELRO				

6c) What **minimal combination** of protections blocks all 4 types of attacks covered in this lab?

6d) **Reflection:** An advanced attacker has techniques such as *information leaks* to bypass ASLR and complex *ROP chains* to bypass NX. Why does **defense in depth** remain necessary despite these bypasses?